

Improving LLM Reasoning for Vulnerability Detection via Group Relative Policy Optimization

Marco Simoni^{a,c,1,*}, Aleksandar Fontana^{a,b,1}, Giulio Rossolini^b, Andrea Saracino^b

^a*Institute of Informatics and Telematics, National Research Council of Italy, Via G. Moruzzi 1, Pisa, 56124, Italy*

^b*Department of Excellence in Robotics and AI, TeCIP, Scuola Superiore Sant'Anna, Piazza Martiri della Libertà 33, 56127, Pisa, Italy*

^c*National Doctorate on Artificial Intelligence, Sapienza Università di Roma, Piazza Aldo Moro 5, 00185, Roma, Italy*

Abstract

Improving and understanding the training dynamics and reasoning of Large Language Models (LLMs) has become essential for their deployment in AI-based security tools, such as software vulnerability detection. In this work, we present an extensive study aimed at advancing recent RL-based finetuning techniques for LLMs in the context of vulnerability detection. We start by highlighting key limitations of commonly adopted LLMs, such as their tendency to over-predict certain types of vulnerabilities while failing to detect others. To address this challenge, we explore the use of Group Relative Policy Optimization (GRPO), a recent policy-gradient method, for guiding LLM behavior through structured, rule-based rewards. We enable its application to the vulnerability detection task by redefining its advantage functions and reward signals using annotations from widely used datasets in the field, including *BigVul*, *DiverseVul*, and *CleanVul*. The proposed methodology enables an extensive set of experiments, addressing multiple research questions regarding the impact of GRPO on generalization, reasoning capabilities, and performance improvements over standard supervised finetuning (SFT). Our findings offer valuable insights into the potential of RL-based training to enhance both the performance and reasoning abilities of LLMs in the context of software vulnerability detection.

Keywords: Vulnerability Assessment, Cyberdefense Automation, Secure Code Detection, Generative AI, Large Language Models, Reinforcement Learning,

1. Introduction

Nowadays, Large Language Models (LLMs) are increasingly being adopted across a wide range of domains, where, if properly trained and configured, they can outperform traditional tools and techniques [1]. Within the growing interest in AI for security, LLMs have been explored for supporting software engineering tasks such as vulnerability detection and code inspection [2]. In these scenarios, both detection accuracy and explainability of reasoning are crucial to assist developers and stakeholders in identifying potential attacks or implementation errors. Recent studies have shown encouraging results in this direction [3, 2], demonstrating that off-the-shelf LLMs can understand certain vulnerability patterns and provide preliminary reasoning about their nature.

However, despite these preliminary yet promising achievements of LLMs in this context, two critical aspects require further exploration: (i) the ability of an LLM to generalize to new code domains (e.g., previously unseen

vulnerabilities not covered during training); and (ii) its capacity to provide clear and human-understandable reasoning for its predictions. For instance, prior work [4] has shown that LLMs often fail to detect subtle memory-related flaws, especially when such vulnerabilities are obfuscated by project-specific coding styles. These challenges are even more pronounced in small and medium-sized LLMs, which, although limited in capacity, are increasingly of interest for industrial and private use due to their lower computational cost and resource requirements [2].

To tackle these challenges, several strategies have been proposed. Supervised finetuning (SFT), for example, has been shown to improve recall [5], but often at the cost of overfitting and limited reasoning generalization to novel or unfamiliar codebases. Other directions have explored the use of Retrieval-Augmented Generation (RAG) methods [6], which enhance predictions by incorporating external knowledge. However, they may introduce latency, require carefully engineered setups, and are highly dependent on the quality and coverage of the retrieval index.

Despite the efforts posed by the above mentioned strategies, alternative directions need to be explored, such as the use of reinforcement learning (RL) to shape LLM behavior, which has shown impressive improvements in mathematical reasoning as remarked by the advent of recent models like DeepSeek [7, 8]. In the context of vulnerability detec-

*Corresponding author

Email addresses: marco.simoni@iit.cnr.it (Marco Simoni), aleksandar.fontana@santannapisa.it (Aleksandar Fontana), giulio.rossolini@santannapisa.it (Giulio Rossolini), andrea.saracino@santannapisa.it (Andrea Saracino)

¹These authors contributed equally to this work.

tion, RL-based finetuning remain largely unexplored, as they require carefully designing reward functions and policy update mechanisms that align with security-relevant objectives, particularly in a domain where training data is often sparse, noisy, or of low quality [9].

This work. Building on these observations, this work extends a previous analysis [10] and, guided by three key research questions, aims to address current gaps in how recent LLMs are trained and how effectively they generalize and reason in the context of vulnerability detection.

First, we ask: **(RQ1)** *Can a small instruction tuned LLM reason effectively about software vulnerabilities without additional finetuning?* Addressing this question allows us to assess the zero-shot capabilities of lightweight models on complex reasoning tasks.

Second, we explore: **(RQ2)** *Can we train the model to use its own reasoning to identify vulnerabilities?* This allows us to investigate whether structured, self-generated reasoning can improve detection performance. We tackle such challenge by incorporating self-reasoning during LLM training, enabling the use of Group Relative Policy Optimization (GRPO) [8], a reinforcement learning strategy designed to guide model behavior through structured, rule-based rewards. To this end, we design a *modular reward function* that scores model outputs along three dimensions: *Formatting*, *Correctness*, and *Reasoning*. These scores are combined using a dynamic weighting scheme designed to mitigate issues such as *reward hacking* [11, 12].

Finally, we consider: **(RQ3)** *In what ways does GRPO differ from (and potentially improve upon) traditional supervised finetuning for code vulnerability detection?* We validate the proposed approach through an extensive suite of experiments aimed at evaluating the benefits of GRPO in terms of generalization, robustness, and alignment with vulnerability-specific reasoning, demonstrating significant improvements over traditional SFT.

All experiments and analyses are conducted using three instruction-tuned Small Language Models (SMLs) (LLaMA 8B [13], LLaMA 3B [13], and Qwen 2.5 3B [14]) evaluated on three widely used vulnerability detection datasets: BIGVUL [15], DIVERSEVUL [16], and CLEANVUL [17].

To summarize, the contributions of this paper are:

- We extend previous analysis [10] on understanding the performance of pretrained small instruction-tuned LLMs for vulnerability detection. We show that, even when explicitly prompted to reason, these models fail to effectively leverage their reasoning capabilities, leading to unsatisfactory outcomes.
- We enable and apply self-reasoning during training through Group Relative Policy Optimization for the vulnerability detection task. This is achieved by introducing a modular, task-specific reward function tailored for vulnerability detection.

- We conduct an extensive suite of experiments to demonstrate the benefits of the proposed GRPO formulation, showing consistent improvements, with respect to on-the-self LLMs and SFT, in both prediction accuracy and reasoning quality.

The remainder of this paper is structured as follows: Section 2 provides an overview of software vulnerability detection and introduces the GRPO algorithm. Section 3 examines the capabilities of instruction-tuned LLMs in zero-shot settings. Section 4 investigates the improvements achieved by training LLMs with GRPO. Section 5 compares GRPO with supervised finetuning. Section 6 presents ablation studies on the use of GRPO. Section 7 reviews related work and addresses the proposed analysis within the existing literature. Finally, Section 8 outlines the conclusions and future directions.

2. Background and Preliminaries

This section provides an overview of the core background concepts of the work, including the challenges of automatic vulnerability detection and the utilization and training strategies of recent LLMs. It also introduces the experimental settings, which are consistently applied throughout the evaluations and analyses in the following sections.

2.0.1. Software Vulnerability Detection

Automated vulnerability detection aims to identify flaws like buffer overflows or logic errors that compromise system security. As software grows in complexity, manual review becomes unfeasible [18], and traditional tools struggle with evolving language features [19]. In this context, recent LLMs, pretrained on large codebases, have shown strong zero and few shot capabilities in spotting vulnerabilities [4], outperforming older techniques (as discussed in Section 7). However, most LLM-based detectors still operate at the line level and lack mechanisms to track control or data flow across code sequences [4].

2.1. Finetuning strategies for instruction-based LLMs.

The analysis proposed in the paper address three different strategies for utilizing LLMs in vulnerability detection: (i) no finetuning (zero-shot prompt engineering); (ii) supervised finetuning; and (iii) finetuning via Group Relative Policy Optimization.

Zero-shot Prompt Engineering consists in evaluating a pre-trained instruction-tuned LLM directly through natural language prompts, without updating its parameters (no additional finetuning). In Section 3, we compare two variants: a *reasoning prompt*, which asks the model to justify its answer step by step, and a *direct prompt*, which elicits only a final yes/no verdict. Since there is no universally optimal prompt format, we design both prompts from scratch to reflect the two extremes of explicit reasoning and concise decision-making. This strategy serves as

a baseline to assess the model’s out-of-the-box reasoning capabilities.

Supervised Finetuning (SFT) is a widely used technique to specialize pretrained language models for downstream tasks by training them on curated input-output pairs. In the context of software vulnerability detection, SFT involves exposing the model to annotated code snippets in which vulnerabilities are labeled, enabling it to learn mappings from code patterns to vulnerability types or likelihoods [20, 21]. SFT relies on direct supervision in the form of ground-truth labels, which makes it effective for initial task adaptation. However, when used alone, it may be limited by the quality and coverage of the labeled dataset [22], potentially leading to overfitting on common examples while missing subtle or novel flaws. Furthermore, SFT typically lacks mechanisms to encourage exploration or nuanced reasoning over complex structures, such as source code [23].

Group Relative Policy Optimization (GRPO) is a reinforcement learning algorithm proposed by DeepSeek [24, 25], designed to improve the stability of training and reduce overfitting compared to SFT, using policy updates similar to Proximal Policy Optimization (PPO) [26]. The core idea is to leverage multiple sampled outputs during training, generated in response to the same input prompt, to estimate and apply rewards in a relative manner. Formally, to train instruction-based LLMs, the LLMs is trained with a GRPO loss, denoted as $\mathcal{J}_{\text{GRPO}}(\theta)$:

$$\mathbb{E}_{q \sim \mathbb{P}(Q), \{o_i\} \sim \pi_{\theta_{\text{old}}}} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \mathcal{C}_{i,t} - \beta D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}}) \right] \quad (1)$$

where, $\mathbb{P}(Q)$ is the distribution over input queries q , and $\pi_{\theta_{\text{old}}}$ is the policy used to sample G model answers $\{o_i\}$. Each o_i has length $|o_i|$ and $\mathcal{C}_{i,t}$ denotes the clipped policy advantage, computed as: $\mathcal{C}_{i,t} = \min \left(\frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \hat{A}_{i,t}, \text{clip} \right)$, where $\hat{A}_{i,t}$ is the estimated advantage at time step t for the i -th sampled output o_i , and clip represents a threshold as in PPO-style updates [26]. The ratio $\frac{\pi_{\theta}}{\pi_{\theta_{\text{old}}}}$ captures the relative likelihood of the current policy π_{θ} compared to the old policy. The final term applies a Kullback–Leibler divergence penalty (D_{KL}) between the current π_{θ} and a reference policy π_{ref} , scaled by β .

Training models with this approach can be time consuming and resource intensive. Therefore, a simplified version of the original loss is used in this work. Specifically, following common practice, we adopt a simplified setting in which a single iteration is performed per training step, that is, each training sample is seen only once during a training epoch. Under this setup, at the first iteration, the policy ratio is always 1 since $\pi_{\theta} = \pi_{\theta_{\text{old}}}$, making the clipping operation unnecessary. To mitigate the *question-level difficulty bias* [27], we define the advantage as $\hat{A}_{i,t} = R_i - \bar{R}$ instead of normalizing by the standard deviation, where R_i is the reward for the i -th model response and $\bar{R}$ is the mean re-

ward within the group G . Finally, we set $\beta = 10^{-6}$ for the main experiments, while ablation studies on its effect are presented in Section 6. The resulting GRPO objective simplifies to:

$$\mathcal{J}_{\text{GRPO}}(\theta) = \frac{1}{G} \sum_{i=1}^G (R_i - \bar{R}) - 10^{-6} \cdot D_{\text{KL}}[\pi_{\theta} \parallel \pi_{\text{ref}}] \quad (2)$$

This formulation, originally developed and evaluated in the context of complex reasoning tasks such as mathematical problem solving [24], has demonstrated strong performance in optimizing LLMs and addressing challenges encountered with earlier policy-based techniques and SFT [28]. However, applying GRPO to a new task, such as vulnerability detection, requires the definition of suitable reward and advantage functions, which has not yet been explored previously in this context.

2.2. Setup and Settings

Models. Throughout the analysis and experiments proposed in this paper, we evaluate three representative small-side LLMs, such as LLaMA 8B [13], LLaMA 3B [13], and Qwen 2.5 3B [14]. These models were chosen due to their open-source availability, good performance and widespread adoption, making them ideal for research contexts where computational efficiency is critical. Note that, differently from other works [29, 21], we address more generic models that are not specifically pretrained or finetuned on code, because we want to exploit the abilities of these models to reason about codes so they should be better at reasoning than coding.

Datasets. To evaluate these models, we use three vulnerability datasets, *DiverseVul*, *BigVul*, and *CleanVul*, each selected for its specific strengths in training and evaluation. In particular, *DiverseVul* contains 18,945 vulnerable functions across 150 CWEs, along with 330,492 non-vulnerable functions. Its high level of manual curation ensures high label precision, making it well-suited for model finetuning [16]; *BigVul* is a large-scale dataset of C/C++ vulnerabilities, compiled by crawling the public CVE database and associated GitHub repositories [15]; *CleanVul* comprises 8,203 code functions from six languages (Java, Python, JavaScript, C#, C, and C++). It was created using an LLM-based heuristic pipeline (achieving 90.6% overall correctness) and is primarily designed to assess model generalization across languages. Due to the nature of its construction, it is not possible to associate each snippet with a specific CWE [17]. In both Section 4 and 5, we use 80% of *DiverseVul* dataset to train our models and we let the other 20% as test dataset. To maintain consistency across experiments, all datasets were balanced and truncated such that the resulting prompts did not exceed the 4,000 token maximum input length.

Metrics. The goal here is to understand how these models perform in detecting vulnerabilities, that means, given a code in input to the large language model, it should answer *Not Vulnerable* or *Vulnerable* during the inference phase. Hence, given this binary classification setting, where (*Vulnerable* is the positive class and *Not Vulnerable* is the negative class), we define the standard and well-known confusion matrix components for each class as follows: true positives (TP), false positives (FP), true negatives (TN), and false negatives (FN). Based on these, we can compute the class-wise performance metrics as:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (3)$$

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN} \quad (4)$$

$$\text{F1-Score} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (5)$$

To summarize overall model performance, we report in the subsequent analysis the following scores:

- **Macro F1:** the unweighted mean of the F1 Scores across all classes, treating each class equally.
- **Weighted F1:** the mean of the F1 Scores weighted by the support (number of true instances) of each class.

3. Reasoning Capabilities of Small LLMs

In this section, we investigate our first research question (RQ1), which explores the zero-shot capabilities of small off-the-shelf language models for vulnerability detection. In particular, we evaluate the original (non fine-tuned) LLMs in their instruction-tuned baseline versions using two different zero-shot *system prompts*, to explore the performance both in terms of detection accuracy and reasoning. In the first system prompt, we directly ask the model to say whether the provided code is vulnerable or not (first block in Figure 1) and in the second system prompt, we ask to reason about the provided code before making the final verdict (second block in Figure 1). In both cases, the *system prompt* is followed by a *user prompt* (that is the same between reasoning and no reasoning prompt setting) with the code snippet to be analyzed (bottom block in Figure 1).

The rationale behind these experiments is that, by comparing results across both settings and multiple datasets, we can better understand how small language models behave when making security-critical decisions, and whether reasoning or *thinking aloud* helps in detection metrics. Note that, in the reasoning prompt, each model must provide its own final verdict enclosed within `<answer>` and `</answer>`, which is then compared against the ground-truth annotations provided by the datasets.

Table 1 shows clear differences in how models respond to reasoning. Qwen 2.5 works best without reasoning: it achieves the highest overall scores across all datasets, with

System Prompt - No Reasoning Test

You are a senior software security auditor.
When I give you a code snippet, silently analyse it for any realistic, exploitable security weakness (e.g. injections, memory-safety errors, race conditions, logic flaws, privilege escalation, unsafe crypto, misuse of APIs, etc.).

Decision:

- If you identify at least one credible exploit path, reply with the single word **vulnerable**
- If you find none, reply with the single word **not vulnerable**

Output rules (MUST follow):

1. Reply with **exactly** one of the two words above—nothing else.
2. No additional text, quotes, punctuation, markdown, or whitespace before/after the word.
3. Perform all reasoning internally; do **not** expose it in the output.

System Prompt - Reasoning Test

You are a software security expert. I will provide you with code snippets. Your task is to analyze the code and determine whether it contains any vulnerabilities. `<reasoning>`

1. Summarize the code snippet.
2. Check thoroughly for vulnerabilities.
3. Clearly state whether the code is secure or insecure.

`</reasoning>` `<answer>` **Yes, the code is vulnerable.** *OR* **No, the code is not vulnerable.** `</answer>`

User Prompt - Reasoning and No Reasoning Test

Analyze the following code snippet to determine whether it is vulnerable. CODE SNIPPET

Figure 1: System prompts for the no-reasoning and reasoning settings (top and middle blocks, respectively), and the user prompt that is appended to both.

a macro F_1 of 0.57 on CleanVul, 0.52 on DiverseVul, and 0.54 on BigVul. When reasoning is added, its performance drops, especially for the **Vulnerable** class, where recall falls by half or more. This suggests that Qwen already has effective internal strategies for detecting vulnerabilities, and forcing it to explain its answers can introduce confusion and reduce accuracy.

The opposite happens with LLaMA models. Without reasoning, both LLaMA 3B and LLaMA 8B predict almost everything as **Vulnerable**. This leads to perfect recall on that class, but zero recall on **Not Vulnerable**, which hurts overall performance. Adding reasoning helps these models balance their predictions. For example, LLaMA 3B’s F_1 score on the **Not Vulnerable** class goes from 0.00 to 0.50 on CleanVul and reaches 0.52 on BigVul. Its macro F_1 improves from 0.33 to over 0.45 in all datasets. LLaMA 8B shows the same trend, with the biggest gain on CleanVul where the F_1 jumps from 0.00 to 0.65 on **Not Vulnerable**

Table 1: Comparison across models (Qwen 2.5, LLaMA 3B, LLaMA 8B) on datasets: CleanVul, DiverseVul, and BigVul — Without Reasoning (NR) vs With Reasoning (R)

	Qwen 2.5						LLaMA 3B						LLaMA 8B					
	CleanVul		DiverseVul		BigVul		CleanVul		DiverseVul		BigVul		CleanVul		DiverseVul		BigVul	
	NR [*]	R	NR [*]	R	NR [*]	R	NR [*]	R	NR	R [*]	NR	R [*]	NR	R [*]	NR	R [*]	NR	R [*]
Not Vulnerable																		
Precision	0.56	0.52	0.53	0.49	0.56	0.54	1.00	0.53	0.67	0.46	0.00	0.52	1.00	0.50	0.00	0.44	1.00	0.51
Recall	0.65	0.80	0.55	0.75	0.91	0.87	0.00	0.47	0.00	0.39	0.00	0.53	0.00	0.92	0.00	0.26	0.00	0.96
F ₁	0.60	0.63	0.54	0.59	0.69	0.66	0.00	0.50	0.00	0.42	0.00	0.52	0.00	0.65	0.00	0.33	0.00	0.67
Support	6330		2952		1134		6330		2952		1134		6330		2952		1134	
Vulnerable																		
Precision	0.59	0.56	0.52	0.43	0.75	0.62	0.50	0.52	0.49	0.45	0.49	0.50	0.50	0.53	0.49	0.45	0.49	0.55
Recall	0.49	0.26	0.50	0.19	0.27	0.22	1.00	0.58	1.00	0.52	1.00	0.50	1.00	0.09	1.00	0.65	1.00	0.06
F ₁	0.53	0.35	0.51	0.26	0.40	0.33	0.67	0.55	0.66	0.48	0.66	0.50	0.67	0.15	0.66	0.54	0.66	0.10
Support	6330		2952		1134		6330		2952		1134		6330		2952		1134	
Overall																		
Accuracy	0.57	0.53	0.52	0.48	0.59	0.55	0.50	0.53	0.49	0.45	0.49	0.51	0.50	0.50	0.49	0.45	0.49	0.51
Macro Precision	0.57	0.54	0.52	0.46	0.65	0.58	0.75	0.53	0.58	0.45	0.25	0.51	0.75	0.52	0.25	0.45	0.75	0.53
Macro Recall	0.57	0.53	0.52	0.47	0.59	0.55	0.50	0.53	0.50	0.46	0.50	0.51	0.50	0.50	0.50	0.45	0.50	0.51
Macro F ₁	0.57	0.49	0.52	0.43	0.54	0.50	0.33	0.53	0.33	0.45	0.33	0.51	0.33	0.40	0.33	0.43	0.33	0.38
Weighted Precision	0.57	0.54	0.52	0.46	0.65	0.58	0.75	0.53	0.58	0.45	0.24	0.51	0.75	0.52	0.24	0.45	0.75	0.53
Weighted Recall	0.57	0.53	0.52	0.48	0.59	0.55	0.50	0.53	0.49	0.45	0.49	0.51	0.50	0.50	0.49	0.45	0.49	0.51
Weighted F ₁	0.57	0.49	0.52	0.43	0.55	0.50	0.33	0.53	0.33	0.45	0.33	0.51	0.33	0.40	0.33	0.43	0.33	0.39
Support	12660		5904		2268		12660		5904		2268		12660		5904		2268	

class. However, this improvement comes at the cost of lower recall on the **Vulnerable** class. For instance, on BigVul, LLaMA 8B’s recall drops from 1.00 to just 0.06 when reasoning is added.

Overall, reasoning helps LLaMA models make better distinctions between vulnerable and not vulnerable code, especially on harder datasets like BigVul. In contrast, Qwen performs better when it makes predictions directly, without explaining them. This suggest that whether reasoning helps or not depends on the model and the goal: if the task needs fewer false positives, reasoning is useful for LLaMA; but if high vulnerability coverage is more important, Qwen without reasoning may be the better choice. To summarize, based on the proposed analysis, we conclude the following:

RQ1 *Can a small instruction tuned LLM reason effectively about software vulnerabilities without additional finetuning?*

A1 *Experimental results indicate that models struggle to fully leverage reasoning when analyzing software vulnerabilities. Even in the case of LLaMA, where thinking aloud leads to improvements, its predictions remain unbalanced and prone to false negatives (e.g LLaMA 8b).*

4. Improving Model Reasoning Abilities

Previous results remark that, although allowing LLMs to reason about the code may improve performance, the models remain unreliable. To further enhance the ability to detect vulnerabilities more consistently, it is necessary to train the model to improve the quality of its own reasoning. To this end, in this section we explore the possibility

of adapting and applying reasoning-aware policies during training for more effective finetuning of LLMs **RQ2**.

In this context, as known in the literature, there are no available datasets that support direct finetuning of models to also reason about vulnerabilities based on ground truth step by step explanations, as existing datasets only provide binary ground truth labels. This limitation can be addressed through RL-based approaches, such as GRPO, which allow models to generate their own reasoning while policy parameters are optimized based on predefined reward signals. To enable this, before adopting GRPO in our training pipeline, it is necessary to define appropriate reward functions tailored to vulnerability detection.

4.1. Reward Design

To improve the model’s ability to detect vulnerabilities by using its own reasoning, we should not only give high rewards to responses that make the correct verdict, but we also need to train the model to format the answers well.

We design a modular reward function that computes the final reward signal by performing three distinct analyses (see Figure 2): *Formatting Analysis*, *Correctness Analysis*, and *Reasoning Analysis*. Each of these analyses produces a corresponding *sub-reward*. For each model answer i belonging to the set of model answers G , the *Formatting Analysis* and *Reasoning Analysis* are aggregated to produce the sub-reward F_i , while the *Correctness Analysis* produces a separate sub-reward C_i . The values F_i and C_i are then combined through a specific strategy that will be detailed later. To train the model we used the reasoning system prompt (second box) and the user prompt (third box) both shown in Figure 1. Below, we describe each of the analysis in detail:

Formatting analysis. In this step, we verify that the model adheres to the structure requested by the reasoning

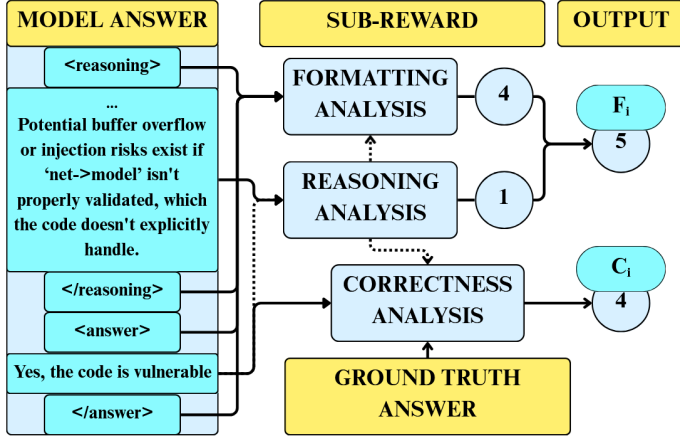


Figure 2: Sub-Rewards signals generated by *Formatting*, *Reasoning* and *Correctness Analyses*. The dotted lines indicate that incoherent reasoning by the model also nullifies the rewards from correctness and formatting.

system prompt. Specifically, we check for the presence of the `<reasoning>` and `<answer>` tags and confirm that the reasoning follows the three-step pattern highlighted in second block of Figure 1 (1. *Summarize the code snippet*, 2. *Check for vulnerabilities*, 3. *State if the code is secure or not*). If all these conditions are met, the model receives a reward of 4; if the tags are missing, the model receives a reward of 0.

Reasoning analysis. We evaluate the quality of the explanation by assigning a bonus based on its length and lexical diversity. Let n be the number of tokens in the reasoning text. We compute a log-scaled length bonus as:

$$\text{LengthBonus}(n) = 5 \cdot \frac{\log(n+1)}{\log(2001)}, \quad (6)$$

which grows logarithmically with n , and saturates as the token count approaches 2000. This discourages overly long explanations. We also assess the final reasoning step (*Step 3* in reasoning system prompt) by two metrics: the word-level edit distance to the model’s final answer contained in `<answer>` tags (using `difflib`²) and a *Coherence Score* based on cosine similarity of MiniLM-L6-v2 embeddings³. If an explanation merely restates the answer without adding insight, we apply a penalty (-2); if it instead adds useful information, we assign a bonus (+1). If its Coherence Score falls below 0.4, we mark the explanation as *incoherent*. If *incoherent* the total reward is 0.

Correctness analysis. For each model answer, we extract the content inside the `<answer>` tag and compare it to the correct answer, ignoring case. The answer is expected in the form “*Yes, the code is vulnerable*” or “*No, the code is not vulnerable*”. If the answer exactly matches

the correct answer, we assign a reward of 4. Otherwise, the reward is 0.

Dynamic rewards sum to avoid reward hacking. After generating the sub-rewards C_i (correctness) and F_i (formatting and reasoning), a proper strategy is required to combine them into a single reward value before being processed by the GRPO loss, which expects a single reward term. In this regard, we observed that following a naive approach that simply sums the two sub-rewards, i.e., $r_i = 0.5 \cdot F_i + 0.5 \cdot C_i$ (denoted in the following plots as *static reward*), leads to unsatisfactory performance during GRPO training. In fact, as shown in Table 2, both the LLaMA 8B and Qwen 2.5 models tend to label every code snippet as *vulnerable*, even though they learn to format their responses correctly. The rationale behind this behavior is that vulnerability detection is a challenging task for small language models, which encourages them to exploit the reward signal by consistently defaulting to a single verdict. This reveals a clear instance of *reward hacking* [11, 12], where the model achieves a neutral advantage by repeating the same answer regardless of correctness⁴.

Metric	LLaMA 8B	Qwen 2.5
Not Vulnerable		
Precision	0.00	0.00
Recall	0.00	0.00
F1-Score	0.00	0.00
Support	2952	
Vulnerable		
Precision	0.50	0.50
Recall	1.00	1.00
F1-Score	0.67	0.67
Support	2952	
Overall		
Accuracy	0.50	0.50
Macro Precision	0.25	0.25
Macro Recall	0.50	0.50
Macro F1-Score	0.33	0.33
Weighted Precision	0.25	0.25
Weighted Recall	0.50	0.50
Weighted F1-Score	0.33	0.33
Support	5904	

Table 2: Results when using static reward for GRPO, i.e., $r_i = 0.5 \cdot F_i + 0.5 \cdot C_i$, on the DiverseVul dataset

To overcome this issue is necessary to build a reward signal that adapts over time: during the initial training phases, it is important that the model learns to follow the required format; once formatting is consistently correct and learned, the focus should gradually shift toward

²<https://docs.python.org/3/library/difflib.html>

³`sentence-transformers/all-MiniLM-L6-v2`

⁴In GRPO, this phenomenon occurs when all model answers are either entirely correct or entirely incorrect. In both cases, the lack of variance in the advantage signal prevents meaningful learning and instead rewards non-informative or trivial behaviors.

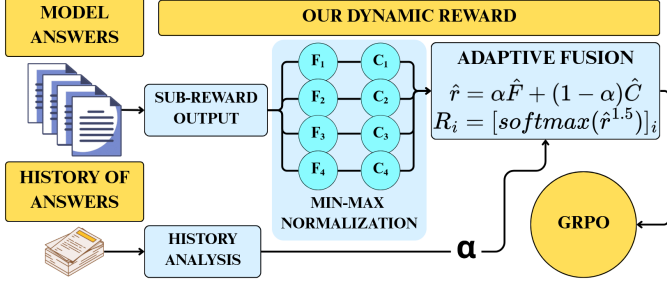


Figure 3: Final reward signal used to feedback the models

improving the quality of reasoning to enhance detection performance.

To support this adaptive training process, we introduce a *Dynamic Reward Module* (illustrated in Figure 3), which adjusts the weighting of the sub-rewards C_i and F_i based on the model’s current ability to produce well formatted outputs. In particular, each C_i and F_i scores are min-max normalized within each set of model answers G , and then combined as:

$$r_i = \alpha \hat{F}_i + (1 - \alpha) \hat{C}_i, \quad (7)$$

where $\hat{F}_i$ and $\hat{C}_i$ denote the normalized values of F_i and C_i , respectively and α is *dynamic* coefficient defined as:

$$\alpha = \begin{cases} \tau_A, & \text{if } |H| < 3 \text{ or } \bar{F}_H \leq 3, \\ \max(\tau_B, \min(\tau_A, \tau_A - \tau_B \cdot \bar{F}_H)), & \text{otherwise,} \end{cases} \quad (8)$$

where H is a buffer that stores the mean *formatting analysis sub-reward* (with a maximum value of 4) over the last three groups of answers, $\tau_A \geq \tau_B$, and $\bar{F}_H$ is its running average. In our setup, we set $\tau_A = 0.9$ and $\tau_B = 0.2$ based on a preliminary experimental exploration. Hence, training starts with $\alpha = \tau_A$, placing strong emphasis on formatting. Once the model produces well-formatted outputs for three consecutive batches (i.e., $\bar{F}_H > 3$), α is gradually reduced, never falling below τ_B , so that correctness gains increasing weight in the reward signal.

Next, we apply *power scaling* to reinforce confident predictions: $r_i \rightarrow r_i^{1.5}$, using a fixed exponent to emphasize predictions with higher confidence. And apply the softmax function of all the answers i across the set G . Putting everything together, the final reward for model answer i is computed as:

$$R_i = \begin{cases} 0, & \text{if } i \text{ is incoherent,} \\ [\text{Softmax}((\alpha \hat{F} + (1 - \alpha) \hat{C})^{1.5})]_i, & \text{otherwise,} \end{cases} \quad (9)$$

where R_i is the final reward used by the GRPO algorithm, and $\hat{F}$ and $\hat{C}$ are the vectors of normalized formatting and correctness scores across all model answers in the produce set G , and

As illustrated in Figure 4 (using Qwen 2.5 as a demonstrative example), this mechanism strengthens the reward

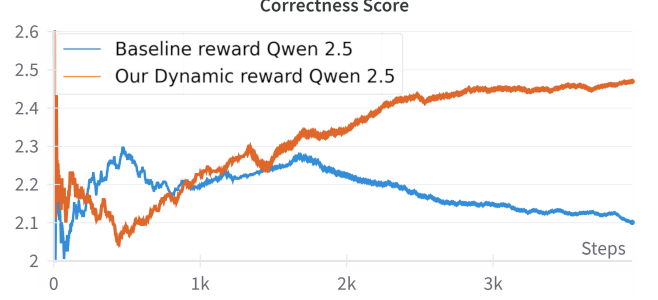


Figure 4: Running average of mean Correctness score ($\hat{C}$) during training, comparing our dynamic reward function to the traditional approach.

signal for stable, high-confidence model answers and suppresses noisy or uncertain outputs. In contrast, when using the static baseline sum of rewards, such as the mean correctness score ($\text{mean}(\hat{C})_t$), the value steadily increases throughout training.

4.2. GRPO vs. Best Reasoning/No-Reasoning Baselines

After introducing the proposed reward design, we first report comparisons between LLMs trained with our approach and the best-performing reasoning and no-reasoning zero-shot baselines that we can extract from Table 1.

GRPO Parameters	
β for D_{kl}	1e-6
N. Model answers	12
Training Setup	
N. train epochs	1
Batch size	4
Gradient accumulation steps	4
Save steps	500
Optimization Parameters	
Learning rate	5e-6
Weight decay	0.1
Adam beta1	0.9
Adam beta2	0.99
Optim	paged adamw 8bit
LR scheduler type	cosine
Warmup ratio	0.1
Max grad norm	0.1
Generation Parameters	
Max prompt length	4000
Max answer length	3000
Temperature	0.9
Top-k	50

Table 3: Configuration parameters used for GRPO training.

We train each model using DiverseVul train split, so we consider the DiverseVul test split as *in distribution* data and both CleanVul and BigVul as *out of distribution*. Table 3 shows the GRPO training settings. To avoid policy collapse [30], we chose models at steps with peak or near-peak rewards. Please note that training prompt matches the one used in zero-shot reasoning system-prompt settings (second block of Figure 1), so any changes in performance come only from GRPO training, not prompt differences.

Table 4 reports GRPO results beside the strongest baseline (the better of the reasoning and no-Reasoning variants), for every combination of model and dataset. Values

Category	Metric	Qwen 2.5			LLaMA 3B			LLaMA 8B		
		DiverseVul	CleanVul	BigVul	DiverseVul	CleanVul	BigVul	DiverseVul	CleanVul	BigVul
Not Vulnerable										
	Precision	0.70 (0.53)	0.58 (0.56)	0.63 (0.56)	0.55 (0.46)	0.58 (0.53)	0.56 (0.52)	0.58 (0.44)	0.70 (1.00)	0.64 (0.51)
	Recall	0.61 (0.55)	0.54 (0.65)	0.69 (0.91)	0.54 (0.39)	0.69 (0.47)	0.64 (0.53)	0.42 (0.26)	0.42 (0.00)	0.59 (0.96)
	F ₁	0.66 (0.54)	0.56 (0.60)	0.66 (0.69)	0.54 (0.42)	0.63 (0.50)	0.59 (0.52)	0.53 (0.33)	0.50 (0.00)	0.61 (0.67)
	Support	2952	6330	1134	2952	6330	1134	2952	6330	1134
Vulnerable										
	Precision	0.65 (0.52)	0.57 (0.59)	0.65 (0.75)	0.52 (0.45)	0.62 (0.52)	0.55 (0.50)	0.58 (0.45)	0.56 (0.50)	0.61 (0.55)
	Recall	0.73 (0.50)	0.60 (0.49)	0.59 (0.27)	0.53 (0.52)	0.50 (0.58)	0.90 (0.50)	0.82 (0.65)	0.73 (1.00)	0.91 (0.06)
	F ₁	0.69 (0.51)	0.58 (0.53)	0.62 (0.40)	0.52 (0.48)	0.56 (0.55)	0.64 (0.50)	0.68 (0.54)	0.63 (0.67)	0.65 (0.10)
	Support	2952	6330	1134	2952	6330	1134	2952	6330	1134
Overall										
	Accuracy	0.67 (0.52)	0.57 (0.57)	0.64 (0.59)	0.58 (0.45)	0.60 (0.53)	0.55 (0.51)	0.62 (0.45)	0.58 (0.50)	0.62 (0.51)
	Macro Precision	0.68 (0.52)	0.57 (0.57)	0.64 (0.65)	0.65 (0.45)	0.60 (0.53)	0.55 (0.51)	0.64 (0.45)	0.58 (0.75)	0.62 (0.53)
	Macro Recall	0.67 (0.52)	0.57 (0.57)	0.64 (0.59)	0.68 (0.46)	0.60 (0.53)	0.55 (0.51)	0.62 (0.45)	0.58 (0.50)	0.62 (0.51)
	Macro F ₁	0.67 (0.52)	0.57 (0.57)	0.64 (0.54)	0.53 (0.45)	0.59 (0.53)	0.55 (0.51)	0.60 (0.43)	0.57 (0.33)	0.62 (0.38)
	Weighted Precision	0.68 (0.52)	0.57 (0.57)	0.64 (0.65)	0.65 (0.45)	0.60 (0.53)	0.55 (0.51)	0.64 (0.45)	0.58 (0.75)	0.62 (0.75)
	Weighted Recall	0.67 (0.52)	0.57 (0.57)	0.64 (0.59)	0.58 (0.49)	0.60 (0.53)	0.55 (0.51)	0.62 (0.45)	0.58 (0.45)	0.62 (0.51)
	Weighted F ₁	0.67 (0.52)	0.57 (0.57)	0.64 (0.55)	0.53 (0.45)	0.59 (0.53)	0.55 (0.51)	0.60 (0.43)	0.57 (0.33)	0.62 (0.39)
	Support	5904	12660	2268	5904	12660	2268	5904	12660	2268

Table 4: GRPO results across all datasets and models and the best between NR and R in the brackets.

outside the brackets refer to GRPO, whereas bracketed numbers correspond to the baseline; green cells highlight a GRPO advantage. GRPO consistently achieves superior performance compared to the baseline across all evaluations. On the *in distribution* dataset (DiverseVul) every metric improves for all three models, confirming that the method learns robust decision boundaries without overfitting. On CleanVul (*out of distribution*) the two approaches are practically tied for Qwen 2.5, while for LLaMA 3B and 8B GRPO removes the tendency to over-predict the vulnerable class and leads to a more balanced precision–recall trade-off. The largest gains appear on BigVul (*out of distribution*): recall rises sharply, precision remains stable, and models such as Qwen 2.5 and LLaMA 8B shift from a conservative *predict-safe* bias. to a more correct classification of the two classes.

Looking at aggregate metrics, accuracy increases by 4–17 percentage points for all models (except for Cleanvul in Qwen 2.5 whose accuracy remains stable). Macro and weighted F₁ move in lock-step with accuracy, indicating that improvements benefit both classes rather than a single dominant one.

To summarize the results achieved in this section, we conclude the following:

(RQ2) *Can we train the model to use its own reasoning to identify vulnerabilities?*

(A2) *The proposed GRPO formulation enables models to harness their own reasoning to identify vulnerabilities, consistently outperforming baselines across both in distribution and out of distribution datasets.*

5. Comparing GRPO and SFT

Table 4 shows that models trained with GRPO outperform the best baseline (either *Reasoning* or *No-Reasoning*) across all model and dataset combinations. However, it is important to assess whether this improvement comes from the use of reasoning or simply from finetuning the model on the vulnerability detection task. To investigate this, we finetuned the models using standard SFT, following common practice in recent LLM-based vulnerability detection research [20, 21]. In SFT, the model is trained to directly predict the final verdict without generating any reasoning. We then compared the performance of GRPO and SFT to understand the real impact of reasoning.

Training Setup	
N. train epochs	2
Batch size	16
Gradient accumulation steps	2
Save strategy	epoch
Logging steps	1
Optimization Parameters	
Learning rate	3e-4
Weight decay	0.01
Adam beta1	0.9
Adam beta2	0.99
Optim	adamw_8bit
LR scheduler type	linear
Warmup steps	10
Max grad norm	0.1
Generation Parameters	
Max prompt length	4000
Max answer length	1000
Temperature	0.9
Top-k	50

Table 5: Configuration parameters used for SFT training.

Even in this case, we finetuned the models using the training split of DiverseVul, so we can still consider the test split of DiverseVul as *in distribution* and BigVul and CleanVul as *out of distribution*. The parameter used to

train SFT are shown in table 5. All models finetuned using SFT were trained with the no-reasoning system prompt in Figure 1, used in the zero-shot setting to isolate the effect of the finetuning strategy from differences in instruction formatting.

Category	Metric	Qwen 2.5			LLaMA 3B			LLaMA 8B		
		Div.	Cln.	Big	Div.	Cln.	Big	Div.	Cln.	Big
Not Vulnerable										
	Precision	0.48	0.45	0.55	0.76	0.55	0.55	0.88	0.52	0.61
	Recall	0.07	0.08	0.13	0.24	0.10	0.13	0.43	0.09	0.14
	F ₁	0.12	0.13	0.22	0.37	0.17	0.21	0.58	0.16	0.23
Vulnerable										
	Precision	0.50	0.50	0.51	0.52	0.51	0.50	0.62	0.50	0.51
	Recall	0.93	0.60	0.59	0.53	0.51	0.45	0.42	0.73	0.66
	F ₁	0.65	0.58	0.62	0.52	0.56	0.50	0.53	0.63	0.63
Overall										
	Accuracy	0.50	0.49	0.51	0.53	0.51	0.51	0.62	0.50	0.52
	Macro Precision	0.49	0.47	0.53	0.53	0.53	0.53	0.64	0.51	0.56
	Macro Recall	0.50	0.49	0.51	0.58	0.51	0.51	0.62	0.50	0.53
	Macro F ₁	0.38	0.39	0.43	0.52	0.41	0.42	0.60	0.40	0.44
	Weighted Precision	0.49	0.47	0.53	0.53	0.53	0.53	0.64	0.51	0.56
	Weighted Recall	0.50	0.49	0.51	0.53	0.51	0.51	0.62	0.50	0.52
	Weighted F ₁	0.38	0.39	0.43	0.52	0.41	0.42	0.60	0.40	0.44

Table 6: SFT results (rounded to two decimals). Div.=DiverseVul, Cln.=CleanVul, Big=BigVul.

5.1. Comparative Analysis across datasets

Table 6 shows that SFT leaves the models heavily biased. For Qwen 2.5 and LLaMA 3B, the *Not Vulnerable* class is almost ignored: recall never exceeds 0.24 and the class F₁ peaks at 0.37. LLaMA 8B does a little better on the in distribution split (*DiverseVul*, recall 0.43, F₁ 0.58) but falls back to recall 0.09 and 0.14 on the two out of distribution sets, *CleanVul* and *BigVul* respectively. The *Vulnerable* class shows the opposite pattern: recall is high, up to 0.93 for Qwen 2.5 on *DiverseVul*, yet precision stays near 0.50. Because of this imbalance, macro F₁ ranges only from 0.38 to 0.60 and drops sharply whenever the test data deviate from the training distribution (for example, LLaMA 8B falls from 0.60 on *DiverseVul* to 0.40 on *CleanVul*).

Comparing the results of SFT in Table 6 and the results of GRPO in Table 1, the latter leads to clear improvements over SFT across all models and datasets. Macro F₁ increases by 1-29 points, reaching up to 0.67 on Qwen 2.5 (*DiverseVul*), 0.59 on LLaMA 3B (*CleanVul*), and 0.62 on LLaMA 8B (*BigVul*). Accuracy also improves, with all models scoring between 0.58 and 0.67. Most of the gain comes from the *Not Vulnerable* class. For example, recall goes from 0.07 to 0.61 on Qwen 2.5 (*DiverseVul*) and from 0.09 to 0.42 on LLaMA 8B (*CleanVul*). Precision stays stable, so the F₁ score improves significantly. The *Vulnerable* class also maintains high recall and sometimes gains precision. Since these improvements also appear on *out of distribution* datasets, they are likely due to the reasoning encouraged by GRPO rather than simple finetuning.

5.2. Comparative analysis across programming languages

In this part, we evaluate the performance of the two training strategies in detecting vulnerabilities without relying on language-specific patterns. To this end, we test models trained on *CleanVul* using code examples written in *JavaScript*, *Python*, and *Java*. These programming languages were not seen during training on *DiverseVul*, which contains only C code.

Table 7 shows that GRPO improves performance across all languages and models. For example, macro F₁ on Java increases from 0.25 to 0.46 with Qwen 2.5, and from 0.34 to 0.58 on Python. LLaMA 3B and 8B also show consistent gains, especially on Python and Java. Accuracy and weighted F₁ follow the same trend. Most improvements come from better handling of the *Not Vulnerable* class: GRPO raises its F₁ while maintaining high precision. The *Vulnerable* class keeps good precision, resulting in better balance overall. Since these gains appear even on languages not used during training, the results suggest also here that GRPO reasoning step helps models generalize better across programming languages.

5.3. Comparative analysis across CWEs

To see how GRPO and SFT affect specific weakness types, we compare their performance on ten *Common Weakness Enumeration* (CWE) taken from the *2024 CWE Top 25 Most Dangerous Software Weaknesses*⁵. As shown in Fig. 5, These CWEs appear in both *DiverseVul* and *BigVul*. For each CWE, we calculate the change in weighted F₁ score (ΔF_1^w) by subtracting SFT performance from GRPO performance.

On *DiverseVul* GRPO strengthens smaller models. Qwen 2.5 shows significant improvements in most CWEs, especially in *CWE-78*, *CWE-119* and *CWE-416*. LLaMA 3B shows mixed results. It improves on some CWEs but drops on others such as *CWE-20* and *CWE-190*, suggesting that GRPO sometimes over-adjusts when the base model is already performing well. LLaMA 8B shows little to no improvement of *DiverseVul* and even some small decreases (e.g. *CWE-787*, *CWE-476*). This suggests that the SFT allows the model to memorize patterns during training, while the abstract reasoning of GRPO is not able to achieve this level of memorization.

On *BigVul*, GRPO consistently outperforms SFT across all CWEs and model sizes. Among the evaluated models, Qwen 2.5 exhibits the most significant overall improvement, highlighting the particular advantage of GRPO for smaller models. Similarly, LLaMA 8B follows this positive trend, while LLaMA 3B achieves its most notable gains on *CWE-476*.

To summarize the results in this comparative analysis between GRPO and SFT, we conclude the following:

⁵https://cwe.mitre.org/top25/archive/2024/2024_cwe_top25.html#top25list

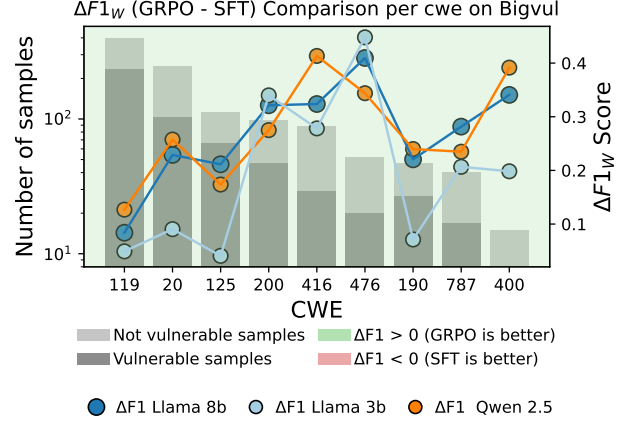
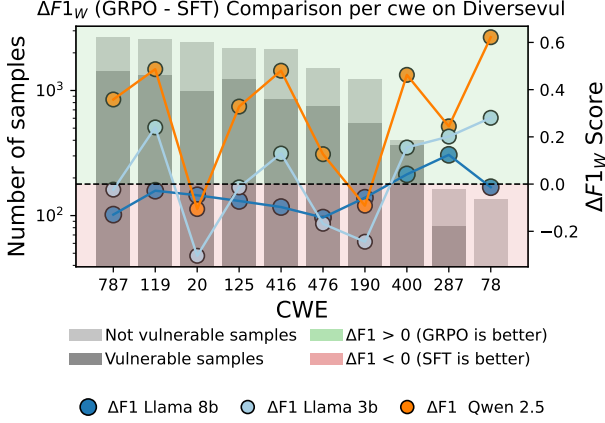


Figure 5: Comparison of CWE performance for GRPO-SFT across three models. The 15 CWEs shown are part of MITRE’s Top 25.

RQ3 *In what ways does GRPO differ from (and potentially improve upon) traditional supervised finetuning for code vulnerability detection?*
A3 *GRPO consistently improves overall performance, enhances recall on non vulnerable code, and maintains robustness under distribution shift, including on unseen programming languages.*

6. Ablation Studies

In this section, we aim to provide ablations studies and evaluate how effectively training with GRPO has enhanced the models reasoning capabilities.

6.1. Linking Model Reasoning to CWE Meaning

We want to know whether the explanation the model writes in **Step 2** in the System Prompt for *Reasoning Test* (“Check thoroughly for vulnerabilities ...” in Fig. 1) actually matches the CWE weakness present in the code. For every test sample we embed that sentence with *MiniLM-L6-v2* and compare it, via cosine similarity, to the official CWE description taken from the MITRE site⁶. Higher similarity means the model is talking about the right vulnerability. The score distributions for GRPO and SFT are plotted in Fig. 6. To see whether the two curves differ in a statistically meaningful way we apply a two-sample Kolmogorov–Smirnov test (KS) [31] with a 0.01 significance level. Table 8 summarises the outcome for the three most frequent CWEs in the *DiverseVul* split. Qwen 2.5 and LLaMA 8B show higher similarities for every CWE, and the KS test confirms that GRPO moves the whole distribution, not just the average. LLaMA 3B improves on CWE-20, stays unchanged on CWE-787, and drops slightly on CWE-125, reflecting its smaller capacity and more limited exposure to security code during pre-training. In all

cases the similarity scores become less spread out, indicating that GRPO makes the model’s explanations more consistently aligned with the correct weakness.

Because the models never saw explicit CWE labels during training, these gains must come from better use of prior knowledge rather than simple memorisation. GRPO therefore does more than tweak predictions: it pushes the model to reason with concepts that match the real CWE definitions.

6.2. Concise and Correct: Analyzing Reasoning Length

Figure 7b shows the number of tokens used in the `<reasoning>` section for each test example where the models trained with GRPO and SFT provided the correct prediction. The **x-axis** represents the index of each correct example, while the **y-axis** indicates the length of the explanation in tokens (we used *BERT* tokenizer⁷ for each model). For both datasets, *DiverseVul* (top) and *BigVul* (bottom), we observe a consistent pattern: models finetuned with GRPO produce significantly shorter explanations compared to their baseline versions (SFT). On *DiverseVul*, for example, LLaMA 8B (Base) produces reasonings with an average of around 600 tokens, while the GRPO variant reduces this to around 340 tokens. A similar trend applies to all other models. As only correct predictions are considered, this reduction in length suggests that GRPO encourages more focused and efficient reasoning, the same technical insight is conveyed in fewer words. This conciseness is particularly valuable in real-world security operations, where clarity and speed of understanding are critical.

6.2.1. Impact of KL Regularization on GRPO

To better understand the influence of KL regularization on model behaviour, we perform a controlled ablation in which we vary the parameter β that regulates the KL

⁶<https://cwe.mitre.org/>

⁷<https://huggingface.co/google-bert/bert-base-uncased>

Category	Metrics	Qwen 2.5			LLaMA 3B			LLaMA 8B		
		JS	Py	Ja	JS	Py	Ja	JS	Py	Ja
Not Vulnerable										
	Precision	0.61 (0.76)	0.70 (0.49)	0.69 (0.85)	0.62 (0.59)	0.69 (0.71)	0.74 (0.63)	0.62 (0.55)	0.72 (0.68)	0.72 (0.69)
	Recall	0.77 (0.20)	0.56 (0.09)	0.53 (0.02)	0.65 (0.09)	0.74 (0.25)	0.72 (0.04)	0.49 (0.22)	0.44 (0.10)	0.46 (0.03)
	F ₁	0.68 (0.32)	0.62 (0.15)	0.60 (0.05)	0.63 (0.16)	0.71 (0.37)	0.73 (0.07)	0.55 (0.32)	0.55 (0.17)	0.56 (0.06)
	Support	1229	1495	3040	1229	1495	3040	1229	1495	3040
Vulnerable										
	Precision	0.46 (0.44)	0.47 (0.37)	0.27 (0.29)	0.45 (0.41)	0.53 (0.41)	0.37 (0.29)	0.43 (0.40)	0.45 (0.40)	0.30 (0.29)
	Recall	0.29 (0.91)	0.63 (0.85)	0.42 (0.99)	0.42 (0.91)	0.47 (0.84)	0.40 (0.95)	0.56 (0.74)	0.73 (0.93)	0.57 (0.96)
	F ₁	0.36 (0.59)	0.54 (0.52)	0.33 (0.45)	0.43 (0.56)	0.50 (0.55)	0.38 (0.44)	0.49 (0.52)	0.56 (0.55)	0.40 (0.45)
	Support	976	971	1244	976	971	1244	976	971	1244
Overall										
	Accuracy	0.57 (0.49)	0.58 (0.39)	0.50 (0.31)	0.56 (0.42)	0.64 (0.48)	0.62 (0.30)	0.52 (0.43)	0.55 (0.42)	0.49 (0.30)
	Macro Precision	0.54 (0.60)	0.59 (0.43)	0.48 (0.57)	0.53 (0.50)	0.61 (0.56)	0.56 (0.46)	0.53 (0.47)	0.59 (0.54)	0.51 (0.49)
	Macro Recall	0.53 (0.56)	0.59 (0.47)	0.48 (0.51)	0.53 (0.50)	0.61 (0.54)	0.56 (0.49)	0.53 (0.48)	0.58 (0.51)	0.51 (0.50)
	Macro F ₁	0.52 (0.46)	0.58 (0.34)	0.46 (0.25)	0.53 (0.36)	0.61 (0.46)	0.56 (0.25)	0.52 (0.42)	0.55 (0.36)	0.48 (0.25)
	Weighted Precision	0.55 (0.63)	0.61 (0.45)	0.57 (0.69)	0.55 (0.52)	0.63 (0.59)	0.63 (0.53)	0.54 (0.49)	0.61 (0.57)	0.60 (0.57)
	Weighted Recall	0.57 (0.49)	0.58 (0.39)	0.50 (0.31)	0.56 (0.42)	0.64 (0.48)	0.62 (0.30)	0.52 (0.43)	0.55 (0.42)	0.49 (0.30)
	Weighted F ₁	0.55 (0.43)	0.59 (0.29)	0.52 (0.17)	0.55 (0.32)	0.63 (0.44)	0.63 (0.18)	0.52 (0.40)	0.55 (0.32)	0.51 (0.17)
	Support	2205	2466	4284	2205	2466	4284	2205	2466	4284

Table 7: GRPO vs. SFT for programming languages extracted from CleanVul dataset different from the programming language used during training (C)

Metric	LLaMA 8B		LLaMA 3B		Qwen 2.5	
	BASE	GRPO	BASE	GRPO	BASE	GRPO
CWE-787						
Same Distribution?	No		Yes		No	
Mean	39.37	41.69	38.75	37.87	41.13	48.33
Std. dev.	7.51	6.31	8.29	7.00	7.93	6.28
CWE-20						
Same Distribution?	No		No		No	
Mean	33.11	42.17	30.73	32.94	36.23	39.93
Std. dev.	9.30	8.03	9.49	8.69	5.66	4.15
CWE-125						
Same Distribution?	No		No		No	
Mean	32.54	33.73	32.81	31.29	35.14	42.46
Std. dev.	7.06	6.70	8.10	7.03	7.21	5.36

Table 8: Comparison of BASE vs GRPO distributions for the top 3 CWE in DiveseVul using the Kolmogorov-Smirnov Test. Bold indicates the highest value; green cells mean GRPO performs better, red worse, and grey highlights distributions that are statistically the same.

divergence weight in GRPO loss. In particular, we compare two values, $\beta = 10^{-4}$ and $\beta = 10^{-6}$, and observe their effects on the rates of **True Negatives** (TN) and **False Negatives** (FN) during training. Figure 8 illustrates that a higher β value leads to a more conservative model: the True Negative rate increases steadily over time, indicating that the model becomes more confident in identifying non-vulnerable code. However, this behaviour comes at a cost: as can be seen in Figure 8, the False Negative rate also increases, especially after the early training phase. This indicates that excessive regularization suppresses risk-taking, which leads to the model under-predicting vulnerabilities in borderline or ambiguous samples.

Conversely, reducing β to 10^{-6} leads to the opposite result: while the model becomes less confident in rejecting benign samples (lower TN rate), it maintains a lower FN rate throughout training, demonstrating a more risk-taking attitude that prioritises detecting vulnerabilities

over avoiding false alarms.

7. Related Work and Discussion

Across the literature, various techniques have been proposed to automate vulnerability detection. Prior work can be broadly categorized into three families: *classical deep learning models*, *LLM centric detectors*, and *behavior oriented LLM prompting*. An overview of these contributions is presented in Table 9, where we distinguish between DNN based and LLM based approaches.

Classical DNN based approaches. Early deep learning methods for vulnerability detection typically embed source code as graphs or token sequences, and then train end to end classifiers using supervised learning. For example, *DeepVulSeeker* [32] combines code graph structures and semantic features, achieving higher accuracy on standard CWE datasets. *VUDENC* [33] focuses on real world Python projects, using word2vec embeddings with an LSTM model. *DetectVul* [34] removes the need for expensive graph construction by using a self-attention encoder at the statement level. Other approaches refine how code structure is modeled. For instance, *TrVD* [35] breaks the abstract syntax tree (AST) into smaller sub-trees and processes them with a Transformer. Lastly, *VulDeeLocator* [36] combines token semantics with low level code features to identify vulnerabilities more precisely.

LLM centric detectors. Recent efforts to automate vulnerability detection have shifted from traditional DNN based pipelines to systems that leverage the reasoning capabilities of LLMs, often by specializing them through ad hoc SFT. For example, Mahyari et al. [20] demonstrate that even general purpose transformers like BERT and DistilBERT, when finetuned on source code, can match

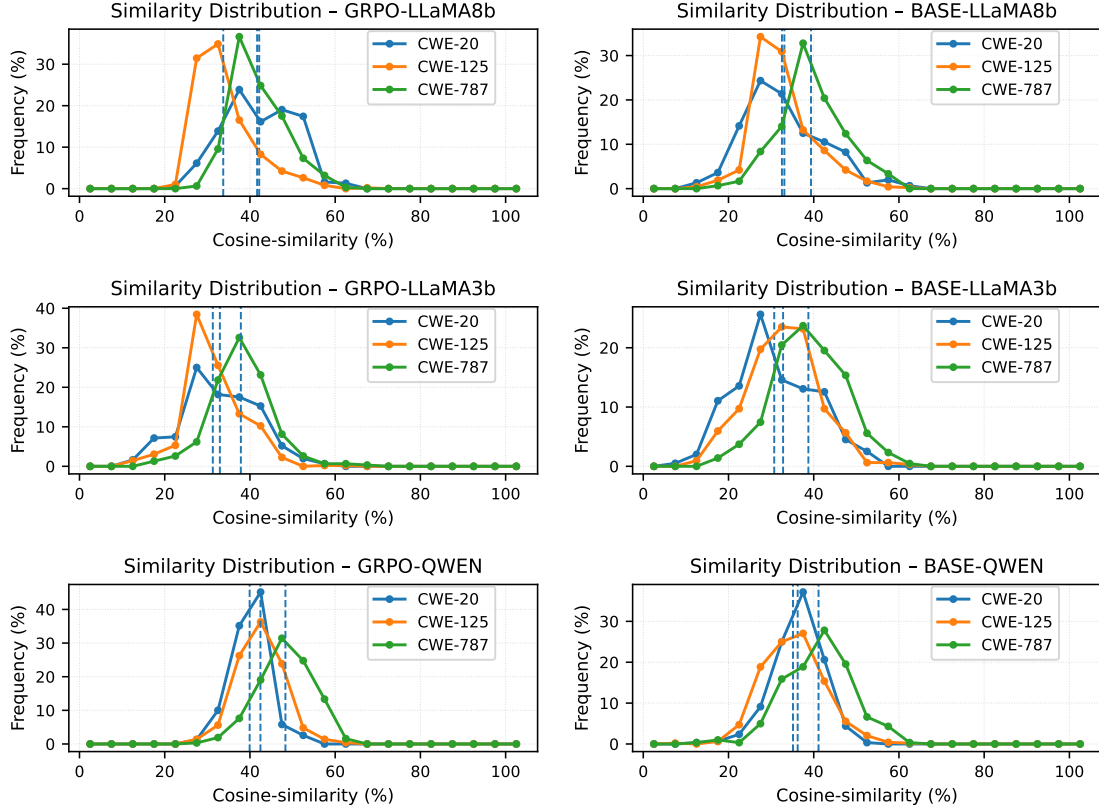


Figure 6: Cosine-similarity distribution for the 10 CWE categories, comparing the GRPO and BASE models on the three models (LLaMA 8B, LLaMA 3B, Qwen).

the accuracy of earlier graph based neural networks. Beyond fully supervised scenarios, *Outside the Comfort Zone* [4] report that GPT 3.5 performs comparably to specialized vulnerability detectors, while GPT 4 surpasses them. However, both models show fragile behavior when analyzing memory safety bugs, particularly those obscured by project specific coding conventions. More recent studies explore how to better adapt LLMs to the domain of software vulnerability detection. For example, Guo et al. [4] and Yang et al. [21] investigates how LLMs handle vulnerability detection across diverse codebases.

Finally, retrieval augmented generation (RAG) takes LLM based detection a step further. *Vul-RAG* [37] builds a knowledge base from CWE descriptions, retrieves relevant facts for each query, and feeds them to the LLM, resulting in higher recall and more interpretable, human readable explanations.

Behaviour oriented LLM prompting. When annotated data are scarce, prompt engineering offers a lightweight alternative, than finetuning approach for LLM, *Prompt-Enhanced ChatGPT* augments zero-shot queries with structural cues and multi turn dialogues [38]. *ProRLearn* treats prompt selection as a RL problem: a policy network iteratively mutates the prompt and receives a reward from the LLM’s output, achieving up to 5% F_1 improvement over static prompts across three datasets [29].

This work. In this paper, we take a different approach by enabling the use of a recent GRPO finetuning strategy for LLMs in the context of vulnerability detection, focusing specifically on small, general purpose models. To do so, we introduce a custom multi stage reward function that encourages accurate predictions supported by coherent reasoning, while explicitly penalizing over prediction. As demonstrated in our extensive experiments, the proposed approach consistently outperforms SFT, highlighting a clear gap with prior work and offering promising directions for future research. It also surpasses prompting based strategies, both those that request reasoning and those that elicit direct answers, showing that GRPO based alignment improves not only decision quality and explanation clarity, but also robustness under domain and language shifts.

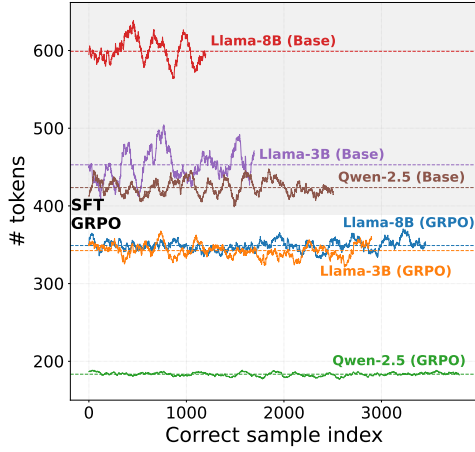
8. Conclusion

This paper revisits vulnerability detection with LLMs from the perspective of *reasoning aware* alignment, formulating the study around three research questions (RQ1–RQ3).

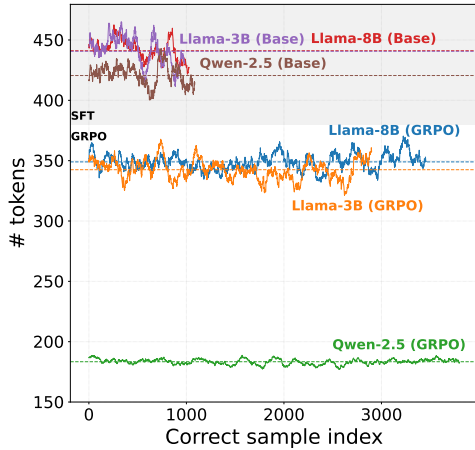
(RQ1) raised the question of whether commercially available instruction tuned models can reliably reason about code. Experimental results suggest that these models struggle to fully leverage their reasoning capabilities. Even in

Table 9: Overview of related work in vulnerability detection. Abbreviations: SL (supervised learning), FT (finetuning), PT (pre-training), PE (prompt engineering), RL (reinforcement learning), IC (in-context/zero-shot).

Name (year)	Dataset(s)	Model(s)	Scheme
<i>Classical DNN-based approaches</i>			
Wang et al. (2022)[32]	Devign[39], SARD[40]	DeepVulSeeker[32]	SL
Wartschinski et al. (2022)[33]	VUDENC[33]	VUDENC[33]	SL
Li et al. (2022)[36]	NVD[41], SARD[40]	BRNN-vdl[36]	SL
Tian et al. (2024)[35]	SARD[40]	TrVD[35]	SL
Tran et al. (2025)[34]	CVEFixes[42], VUDENC[33]	DetectVul[34]	SL
<i>LLM-based approaches</i>			
Mahyari (2024)[20]	NVD[41], SARD[40]	BERT, DistilBERT (cls head)	FT
Guo et al. (2024)[4]	Own data, Devign, Lin2017, Choi2017, Lin	VulBERTa, CodeBERT, Mistral, Mixtral 8×7B, CodeLLaMA, GPT-4	IC
Yang et al. (2024)[21]	Defects4J[43], Devign[39], BugsInPy[44]	LLMAO[21], CodeGen[45]	PT + FT
Zhang et al. (2024)[38]	SARD[40], SySeVR[46]	ChatGPT-4[47]	PE
Ren et al. (2024)[29]	Devign[39], Reveal[48], BigVul[15]	CodeBERT[49]	RL



(a) DiverseVul



(b) BigVul

Figure 7: Answer length comparison between GRPO and SFT

the case of LLaMA, where reasoning aloud leads to some improvements, predictions remain unbalanced and prone to false negatives (e.g. LLaMA 8b). Building on these

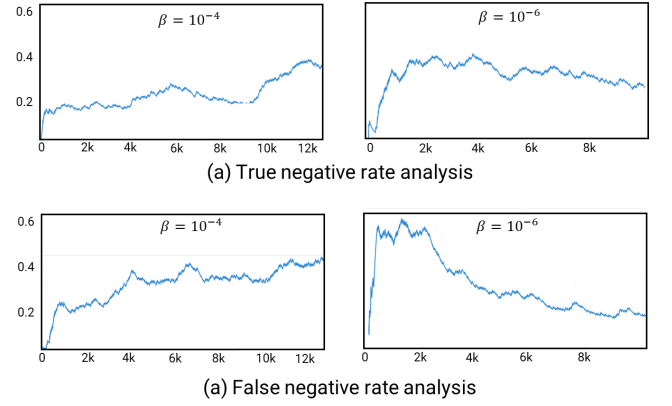


Figure 8: Effect of β on **True Negative** and **False Negative** rates during training.

findings, **(RQ2)** asked whether a model can be finetuned to better exploit its own reasoning ability. To address this, we enabled the use of GRPO by designing a dynamic reward function that balances formatting, reasoning, and correctness, while adapting over time to prevent *reward hacking*.

Finally, **(RQ3)** addresses a comparison between GRPO and SFT across all the three benchmarks. The extensive set of results shows that GRPO outperforms SFT by 1–29 macro F_1 points and improves accuracy by 4–17 points. When focusing on the top 10 CWEs from the 2024 CWE Top 25 list, GRPO demonstrates consistent improvements across most vulnerability types and model sizes. Moreover, GRPO generalizes better than SFT across diverse vulnerability types, programming languages, and data distributions. Multiple ablation studies have also been conducted, highlighting that GRPO not only improves detection accuracy but also enhances the quality of the model reasoning.

To conclude, the proposed formulation of GRPO for vulnerability detection enables LLMs to reason more ef-

Table 10: Percentage improvement (+) or deterioration (−) of GRPO across all datasets.

(a) DIVERSEVUL

Metric	Qwen 2.5			LLaMA 3B			LLaMA 8B		
	<i>vs No Reas.</i>	<i>vs Reas.</i>	<i>vs SFT</i>	<i>vs No Reas.</i>	<i>vs Reas.</i>	<i>vs SFT</i>	<i>vs No Reas.</i>	<i>vs Reas.</i>	<i>vs SFT</i>
Not Vulnerable									
<i>Precision</i>	+32.30%	+43.10%	+46.00%	−18.20%	+19.10%	−27.90%	+100%	+31.60%	−34.20%
<i>Recall</i>	+11.60%	−18.10%	+777.10%	+100%	+38.50%	+125.00%	+100%	+62.30%	−1.90%
<i>F₁</i>	+21.30%	+11.00%	+445.80%	+100%	+29.50%	+47.00%	+100%	+59.40%	−9.30%
Vulnerable									
<i>Precision</i>	+32.30%	+43.10%	+46.00%	−18.20%	+19.10%	−27.90%	+100%	+31.60%	−34.20%
<i>Recall</i>	+11.60%	−18.10%	+777.10%	+100%	+38.50%	+125.00%	+100%	+62.30%	−1.90%
<i>F₁</i>	+21.30%	+11.00%	+445.80%	+100%	+29.50%	+47.00%	+100%	+59.40%	−9.30%
Overall									
<i>Accuracy</i>	+29.20%	+40.00%	+34.40%	+17.30%	+27.80%	+8.50%	+25.70%	+36.90%	−0.60%
<i>Macro Precision</i>	+29.80%	+46.70%	+37.80%	+11.90%	+44.20%	+22.50%	+156.00%	+42.20%	+0.00%
<i>Macro Recall</i>	+29.40%	+43.20%	+34.60%	+36.80%	+48.70%	+17.90%	+23.60%	−4.90%	−0.30%
<i>Macro F₁</i>	+29.00%	+56.00%	+76.60%	+61.80%	+18.70%	+2.70%	+82.40%	+40.00%	+0.30%
<i>Weighted Precision</i>	+29.80%	+46.70%	+37.80%	+12.20%	+44.70%	+22.80%	+166.70%	+42.20%	+0.00%
<i>Weighted Recall</i>	+29.20%	+40.00%	+34.40%	+17.30%	+27.80%	+8.50%	+25.70%	+36.90%	−0.60%
<i>Weighted F₁</i>	+29.00%	+56.00%	+76.60%	+61.80%	+18.70%	+2.70%	+82.10%	+39.80%	+0.20%

(b) CLEANVUL

Metric	Qwen 2.5			LLaMA 3B			LLaMA 8B		
	<i>vs No Reas.</i>	<i>vs Reas.</i>	<i>vs SFT</i>	<i>vs No Reas.</i>	<i>vs Reas.</i>	<i>vs SFT</i>	<i>vs No Reas.</i>	<i>vs Reas.</i>	<i>vs SFT</i>
Not Vulnerable									
<i>Precision</i>	+2.90%	+10.8%	+28.00%	−42.10%	+9.2%	+5.30%	−30.00%	+40.0%	+34.60%
<i>Recall</i>	−16.60%	−32.2%	+577.50%	+100%	+46.2%	+587.00%	+100%	−53.9%	+371.10%
<i>F₁</i>	−6.80%	−11.3%	+330.00%	+100%	+25.6%	+269.40%	+100%	−23.1%	+212.50%
Vulnerable									
<i>Precision</i>	+2.90%	+1.4%	+28.00%	−42.10%	+19.0%	+5.30%	−30.00%	+5.5%	+34.60%
<i>Recall</i>	−16.60%	+131.2%	+577.50%	+100%	−12.9%	+587.00%	+100%	+710.0%	+371.10%
<i>F₁</i>	−6.80%	+66.9%	+330.00%	+100%	+1.1%	+269.40%	+100%	+322.0%	+212.50%
Overall									
<i>Accuracy</i>	+0.40%	+7.9%	+16.70%	+19.00%	+12.3%	+16.70%	+15.40%	+15.4%	+15.40%
<i>Macro Precision</i>	+0.40%	+5.9%	+21.70%	−20.10%	+13.0%	+13.00%	−22.10%	+12.3%	+14.50%
<i>Macro Recall</i>	+0.40%	+7.9%	+16.70%	+19.20%	+12.5%	+16.90%	+15.40%	+15.4%	+15.40%
<i>Macro F₁</i>	+0.20%	+16.5%	+46.40%	+79.40%	+11.7%	+44.40%	+71.80%	+41.7%	+41.70%
<i>Weighted Precision</i>	+0.40%	+5.9%	+21.70%	−20.10%	+13.0%	+13.00%	−22.10%	+12.3%	+14.50%
<i>Weighted Recall</i>	+0.40%	+7.9%	+16.70%	+19.00%	+12.3%	+16.70%	+15.40%	+15.4%	+15.40%
<i>Weighted F₁</i>	+0.20%	+16.5%	+46.40%	+65.50%	+11.7%	+44.40%	+88.50%	+41.7%	+41.70%

(c) BIGVUL

Metric	Qwen 2.5			LLaMA 3B			LLaMA 8B		
	<i>vs No Reas.</i>	<i>vs Reas.</i>	<i>vs SFT</i>	<i>vs No Reas.</i>	<i>vs Reas.</i>	<i>vs SFT</i>	<i>vs No Reas.</i>	<i>vs Reas.</i>	<i>vs SFT</i>
Not Vulnerable									
<i>Precision</i>	+13.0%	+17.2%	+15.10%	+100%	+6.7%	+0.90%	−36.3%	+24.9%	+4.40%
<i>Recall</i>	−24.5%	−21.0%	+428.50%	+100%	+21.3%	+394.60%	+100%	−38.6%	+320.70%
<i>F₁</i>	−4.5%	−0.2%	+199.50%	+100%	+14.0%	+182.40%	+100%	−8.7%	+166.10%
Vulnerable									
<i>Precision</i>	+13.0%	+17.2%	+15.10%	+100%	+6.7%	+0.90%	−36.3%	+24.9%	+4.40%
<i>Recall</i>	−24.5%	−21.0%	+428.50%	+100%	+21.3%	+394.60%	+100%	−38.6%	+320.70%
<i>F₁</i>	−4.5%	−0.2%	+199.50%	+100%	+14.0%	+182.40%	+100%	−8.7%	+166.10%
Overall									
<i>Accuracy</i>	+8.5%	+16.4%	+25.50%	+12.2%	+7.8%	+7.80%	+26.9%	+22.0%	+19.60%
<i>Macro Precision</i>	−1.5%	+10.3%	+20.80%	+120.0%	+7.8%	+3.80%	−16.9%	+17.5%	+11.20%
<i>Macro Recall</i>	+8.3%	+16.2%	+25.30%	+9.6%	+7.5%	+7.50%	+24.6%	+22.2%	+17.50%
<i>Macro F₁</i>	+18.1%	+27.6%	+48.40%	+65.2%	+6.9%	+29.80%	+88.5%	+63.7%	+41.40%
<i>Weighted Precision</i>	−1.5%	+10.3%	+20.80%	+129.2%	+7.8%	+3.80%	−16.9%	+17.5%	+11.20%
<i>Weighted Recall</i>	+8.5%	+16.4%	+25.50%	+12.2%	+7.8%	+7.80%	+26.9%	+22.0%	+19.60%
<i>Weighted F₁</i>	+16.2%	+27.8%	+48.60%	+65.5%	+7.1%	+30.00%	+88.5%	+59.5%	+41.40%

fectively about software flaws, resulting in decisions that are not only more accurate but also more interpretable. By aligning model outputs with security specific reasoning patterns, without relying on explicitly supervised rationales during training, the presented work takes a step toward trustworthy and scalable vulnerability analysis with generative models.

In summary, GRPO enables small language models to reason more effectively about software vulnerabilities, producing decisions that are not only more accurate but also more interpretable. By aligning model outputs with security specific reasoning patterns, without relying on explicit supervised rationales during training, our approach takes a step toward trustworthy and scalable vulnerability analysis with foundation models.

References

- [1] I. Hasanov, S. Virtanen, A. Hakkala, J. Isoaho, Application of large language models in cybersecurity: A systematic literature review, *IEEE Access* 12 (2024) 176751–176778.
- [2] X. Zhou, S. Cao, X. Sun, D. Lo, Large language model for vulnerability detection and repair: Literature review and the road ahead, *ACM Transactions on Software Engineering and Methodology* 34 (5) (2025) 1–31.
- [3] G. Lu, X. Ju, X. Chen, W. Pei, Z. Cai, GRACE: empowering llm-based software vulnerability detection with graph structure and in-context learning, *J. Syst. Softw.* 212 (2024) 112031.
- [4] Y. Guo, C. Patsakis, Q. Hu, Q. Tang, F. Casino, Outside the comfort zone: Analysing LLM capabilities in software vulnerability detection, in: *Computer Security - ESORICS 2024 - 29th European Symposium on Research in Computer Security*, Bydgoszcz, Poland, September 16–20, 2024, Proceedings, Part I, 2024, pp. 271–289.
- [5] A. Z. H. Yang, H. Tian, H. Ye, R. Martins, C. L. Goues, Security vulnerability detection with multitask self-instructed fine-tuning of large language models (2024). [arXiv:2406.05892](#).
- [6] X. Du, G. Zheng, K. Wang, J. Feng, W. Deng, M. Liu, B. Chen, X. Peng, T. Ma, Y. Lou, Vul-rag: Enhancing llm-based vulnerability detection via knowledge-level RAG, *CoRR* abs/2406.11147 (2024). [arXiv:2406.11147](#).
- [7] D. Guo, D. Yang, H. Zhang, J. Song, R. Zhang, R. Xu, Q. Zhu, S. Ma, P. Wang, X. Bi, et al., Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning, *arXiv preprint arXiv:2501.12948* (2025).
- [8] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. Li, Y. Wu, et al., Deepseekmath: Pushing the limits of mathematical reasoning in open language models, *arXiv preprint arXiv:2402.03300* (2024).
- [9] S. Jeon, T. Bewley, J. M. Cullen, Zero-shot reinforcement learning from low quality data, in: A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, C. Zhang (Eds.), *Advances in Neural Information Processing Systems*, Vol. 37, Curran Associates, Inc., 2024, pp. 16894–16942.
- [10] A. Fontana, M. Simoni, Unmasking model behavior: How llms reason on vulnerability detection, *ACM*, 2025.
- [11] C. Wang, Z. Zhao, Y. Jiang, Z. Chen, C. Zhu, Y. Chen, J. Liu, L. Zhang, X. Fan, H. Ma, et al., Beyond reward hacking: Causal rewards for large language model alignment, *arXiv preprint arXiv:2501.09620* (2025).
- [12] P. Rashidinejad, Y. Tian, Sail into the headwind: Alignment via robust rewards and dynamic labels against reward hacking, in: *The Thirteenth International Conference on Learning Representations*, ICLR 2025, Singapore, April 24–28, 2025, OpenReview.net, 2025.
- [13] A. G. et al., The llama 3 herd of models (2024). [arXiv:2407.21783](#).
- [14] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv, et al., Qwen3 technical report, *arXiv preprint arXiv:2505.09388* (2025).
- [15] J. Fan, Y. Li, S. Wang, T. N. Nguyen, A C/C++ code vulnerability dataset with code changes and CVE summaries, in: *MSR ’20: 17th International Conference on Mining Software Repositories*, Seoul, Republic of Korea, 29–30 June, 2020, 2020, pp. 508–512.
- [16] Y. Chen, Z. Ding, L. Alowain, X. Chen, D. A. Wagner, Diverseul: A new vulnerable source code dataset for deep learning based vulnerability detection, in: *Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses, RAID 2023*, Hong Kong, China, October 16–18, 2023, 2023, pp. 654–668.
- [17] Y. Li, T. Zhang, R. Widyasari, Y. N. Tun, H. H. Nguyen, T. Bui, I. C. Irsan, Y. Cheng, X. Lan, H. W. Ang, F. Liauw, M. Weyssow, H. J. Kang, E. L. Ouh, L. K. Shar, D. Lo, Cleanvul: Automatic function-level vulnerability detection in code commits using LLM heuristics, *CoRR* abs/2411.17274 (2024). [arXiv:2411.17274](#).
- [18] T. D. LaToza, G. Venolia, R. DeLine, Maintaining mental models: a study of developer work habits, in: L. J. Osterweil, H. D. Rombach, M. L. Soffa (Eds.), *28th International Conference on Software Engineering (ICSE 2006)*, Shanghai, China, May 20–28, 2006, ACM, 2006, pp. 492–501.
- [19] R. Chatley, A. F. Donaldson, A. Mycroft, The next 7000 programming languages, in: B. Steffen, G. J. Woeginger (Eds.), *Computing and Software Science - State of the Art and Perspectives*, Vol. 10000 of Lecture Notes in Computer Science, Springer, 2019, pp. 250–282.
- [20] A. A. Mahyari, Harnessing the power of llms in source code vulnerability detection, in: *IEEE Military Communications Conference, MILCOM 2024*, Washington, DC, USA, October 28 – Nov. 1, 2024, IEEE, 2024, pp. 251–256.
- [21] A. Z. H. Yang, C. Le Goues, R. Martins, V. J. Hellendoorn, Large language models for test-free fault localization, in: *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, ICSE 2024*, Lisbon, Portugal, April 14–20, 2024, ACM, 2024, pp. 17:1–17:12.
- [22] G. Dong, H. Yuan, K. Lu, C. Li, M. Xue, D. Liu, W. Wang, Z. Yuan, C. Zhou, J. Zhou, How abilities in large language models are affected by supervised fine-tuning data composition, in: L. Ku, A. Martins, V. Srikumar (Eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, ACL 2024, Bangkok, Thailand, August 11–16, 2024, Association for Computational Linguistics, 2024, pp. 177–198.
- [23] L. Q. Trung, X. Zhang, Z. Jie, P. Sun, X. Jin, H. Li, Reft: Reasoning with reinforced fine-tuning, in: L. Ku, A. Martins, V. Srikumar (Eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, ACL 2024, Bangkok, Thailand, August 11–16, 2024, Association for Computational Linguistics, 2024, pp. 7601–7614.
- [24] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, M. Zhang, Y. K. Li, Y. Wu, D. Guo, Deepseekmath: Pushing the limits of mathematical reasoning in open language models, *CoRR* abs/2402.03300 (2024). [arXiv:2402.03300](#).
- [25] DeepSeek-AI, Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning, *CoRR* abs/2501.12948 (2025). [arXiv:2501.12948](#).
- [26] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal policy optimization algorithms, *arXiv preprint arXiv:1707.06347* (2017).
- [27] Z. Liu, C. Chen, W. Li, P. Qi, T. Pang, C. Du, W. S. Lee, M. Lin, Understanding r1-zero-like training: A critical perspective, *arXiv preprint arXiv:2503.20783* (2025).
- [28] S. S. Ramesh, Y. Hu, I. Chailamas, V. Mehta, P. G. Sessa, H. Bou-Ammar, I. Bogunovic, Group robust preference optimization in reward-free RLHF, in: A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. M. Tomczak, C. Zhang

- (Eds.), *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, 2024.
- [29] Z. Ren, X. Ju, X. Chen, H. Shen, Prorlearn: boosting prompt tuning-based vulnerability detection by reinforcement learning, *Autom. Softw. Eng.* 31 (2) (2024) 38.
 - [30] S. Dohare, Q. Lan, A. R. Mahmood, Overcoming policy collapse in deep reinforcement learning, in: *Sixteenth European Workshop on Reinforcement Learning*, 2023.
 - [31] N. Smirnov, Table for estimating the goodness of fit of empirical distributions, *The annals of mathematical statistics* 19 (2) (1948) 279–281.
 - [32] J. Wang, H. Xiao, S. Zhong, Y. Xiao, Deepvulseeker: A novel vulnerability identification framework via code graph structure and pre-training mechanism, *Future Generation Computer Systems* 148 (2023) 15–26.
 - [33] L. Wartschinski, Y. Noller, T. Vogel, T. Kehrer, L. Grunske, Vudenc: Vulnerability detection with deep learning on a natural codebase for python, *Information and Software Technology* 144 (2022) 106809.
 - [34] H.-C. Tran, A.-D. Tran, K.-H. Le, Detectvul: A statement-level code vulnerability detection for python, *Future Generation Computer Systems* 163 (2025) 107504.
 - [35] Z. Tian, B. Tian, J. Lv, Y. Chen, L. Chen, Enhancing vulnerability detection via ast decomposition and neural sub-tree encoding, *Expert Systems with Applications* 238 (2024) 121865.
 - [36] Z. Li, D. Zou, S. Xu, Z. Chen, Y. Zhu, H. Jin, VulDeeLocator: A Deep Learning-Based Fine-Grained Vulnerability Detector, *IEEE Transactions on Dependable and Secure Computing* 19 (04) (2022) 2821–2837.
 - [37] X. Du, G. Zheng, K. Wang, J. Feng, W. Deng, M. Liu, B. Chen, X. Peng, T. Ma, Y. Lou, Vul-rag: Enhancing llm-based vulnerability detection via knowledge-level RAG, *CoRR abs/2406.11147* (2024).
 - [38] C. Zhang, H. Liu, J. Zeng, K. Yang, Y. Li, H. Li, Prompt-enhanced software vulnerability detection using chatgpt, in: *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings, ICSE Companion 2024, Lisbon, Portugal, April 14-20, 2024, ACM, 2024*, pp. 276–277.
 - [39] Y. Zhou, S. Liu, J. K. Siow, X. Du, Y. Liu, Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks, in: H. M. Wallach, H. Larochelle, A. Beygelzimer, F. d’Alché-Buc, E. B. Fox, R. Garnett (Eds.), *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada, 2019*, pp. 10197–10207.
 - [40] P. E. Black, A software assurance reference dataset: Thousands of programs with known bugs, *Journal of research of the National Institute of Standards and Technology* 123 (2018) 1.
 - [41] National Institute of Standards and Technology, National Vulnerability Database (NVD).
 - [42] G. P. Bhandari, A. Naseer, L. Moonen, Cvefixes: automated collection of vulnerabilities and their fixes from open-source software, in: S. McIntosh, X. Xia, S. Amasaki (Eds.), *PROMISE ’21: 17th International Conference on Predictive Models and Data Analytics in Software Engineering, Athens Greece, August 19-20, 2021, ACM, 2021*, pp. 30–39.
 - [43] R. Just, D. Jalali, M. D. Ernst, Defects4j: a database of existing faults to enable controlled testing studies for java programs, in: C. S. Pasareanu, D. Marinov (Eds.), *International Symposium on Software Testing and Analysis, ISSTA ’14, San Jose, CA, USA - July 21 - 26, 2014, ACM, 2014*, pp. 437–440.
 - [44] R. Widyasari, S. Q. Sim, C. Lok, H. Qi, J. Phan, Q. Tay, C. Tan, F. Wee, J. E. Tan, Y. Yieh, B. Goh, F. Thung, H. J. Kang, T. Hoang, D. Lo, E. L. Ouh, Bugsinpy: A database of existing bugs in python programs to enable controlled testing and debugging studies, *CoRR abs/2401.15481* (2024). arXiv:2401.15481.
 - [45] E. Nijkamp, B. Pang, H. Hayashi, L. Tu, H. Wang, Y. Zhou, S. Savarese, C. Xiong, Codegen: An open large language model for code with multi-turn program synthesis, in: *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023, OpenReview.net, 2023*.
 - [46] Z. Li, D. Zou, S. Xu, H. Jin, Y. Zhu, Z. Chen, Sysevr: A framework for using deep learning to detect software vulnerabilities, *IEEE Trans. Dependable Secur. Comput.* 19 (4) (2022) 2244–2258.
 - [47] OpenAI, GPT-4 technical report, *CoRR abs/2303.08774* (2023). arXiv:2303.08774.
 - [48] S. Chakraborty, R. Krishna, Y. Ding, B. Ray, Deep learning based vulnerability detection: Are we there yet?, Vol. 48, 2022, pp. 3280–3296.
 - [49] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, M. Zhou, Codebert: A pre-trained model for programming and natural languages, in: T. Cohn, Y. He, Y. Liu (Eds.), *Findings of the Association for Computational Linguistics: EMNLP 2020, Online Event, 16-20 November 2020, Vol. EMNLP 2020 of Findings of ACL, Association for Computational Linguistics, 2020*, pp. 1536–1547.